KEYBOARDWARRIOR

User Guide

Play, practice and track your progress.
Commands, combat, scoring and saved runs.



ONLINE USER GUIDE

https://ay2627s1-cs2113-w12-4.github.io/tp/UserGuide.html

PRODUCT v2.0

27 October 2026

Contents

Introduction . . . 3

Quick Start . . . 3

Saved typing history . . . 3

Leaderboard . . . 4

Main menu and game difficulty . . . 4

Seeds and repeatable runs . . . 5

Saving and continuing a run . . . 5

Skills and countdown . . . 6

Combat and progression . . . 7

Typing feedback . . . 8

- Words per minute (WPM) . . . 8
- Character accuracy . . . 8
- Damage and the deadline . . . 8
- Example . . . 8

Introduction

Keyboard Warrior is a terminal typing combat game. Choose a skill, type its passage and deal damage based on your WPM and character accuracy. Survive five regular scenarios and both phases of the final Kernel Overlord boss to win. Your starting HP depends on the selected game difficulty and carries between all encounters; there is no healing during a run.

Quick Start

1. Install Java 25.
2. Place the release JAR in a writable folder, open a terminal in that folder and run `java -jar KeyboardWarrior.jar`. The same JAR works on Windows, macOS and Linux.
3. Type `start` and press Enter at the main menu.
4. Enter a seed and press Enter, or press Enter on an empty line for a random seed.
5. Type a displayed skill name, such as `Ping`, and press Enter.
6. Type the passage and press Enter to submit. In a supported terminal, the live countdown can also submit your current text automatically when time expires.

Only Java 25 and the JAR are needed for a release. If you have the source project, you can instead run `sh ./run-game.sh` on macOS/Linux or `.\run-game.bat` in Windows PowerShell. These optional launchers compile through Gradle and start Java directly with assertions enabled.

Use IntelliJ's Terminal or a native terminal for the live timer and capture of typing before Enter. Gradle `run`, IDE Run consoles and automated process launchers may redirect input/output. The game then starts in plain-text mode, where the menu and passages remain visible and the time allowance is printed once. Plain mode waits for Enter, even after the displayed deadline, then scores the submitted text using the actual elapsed time. It cannot reliably capture keystrokes that the console has not delivered before Enter, so automatic submission is unavailable in this mode. On Windows, Windows Terminal is an alternative when an IDE terminal lacks live display.

Type `exit` and press Enter at any phase to quit. End-of-input also closes the game cleanly. To enable development assertions while running the JAR, use:

```
java -ea -jar KeyboardWarrior.jar
```

Saved typing history

At the main menu, type `2` or `records` and press Enter to view the latest ten typing attempts, oldest first. Each entry represents one submitted passage, not a whole game, and shows WPM and accuracy. The `#` column is its position in the displayed history, not a permanent record ID or score rank. The full main menu appears again after the records, where you can enter `start`, `records`, `exit`, `choose difficulty`, `continue` or `leaderboard`.

Every submitted attempt is saved automatically, including partial or empty text captured when the live timer expires in a supported terminal. History survives restarting the application. The file is `data/keyboardwarrior.txt`, relative to the folder from which the game is launched.

If there is no history yet, the menu displays an empty-history message. If the file cannot be read or contains an invalid record, it displays an error. The full menu also reappears after an empty message or loading error, keeping your selected difficulty. A saving error is reported while the current combat turn continues; that attempt may not appear in Records.

Leaderboard

At the main menu, enter `6` or `leaderboard` to view your single all-time highest-scoring attempt. The display shows the WPM and accuracy of one typing attempt, not total combat damage or a whole-game score. This is a local leaderboard shared by attempts across all difficulty modes and seeds in the same game folder. The highest WPM wins, with higher accuracy breaking WPM ties. There is no minimum accuracy requirement. Exact ties leave the saved best unchanged. Values are displayed to two decimal places, but the comparison uses their full precision, so two attempts that look equal may still differ. The high score is saved separately in `data/keyboardwarrior.txt.high-score` and survives restarting the game and removal of older attempts from the ten-entry history. Only a better attempt replaces it. On first use, existing typing history supplies the initial best score; attempts already discarded before this feature was added cannot be recovered. The full main menu appears again after the leaderboard. The chronological Records view remains unchanged. If no attempts are saved, an empty message appears. Unreadable or invalid high-score data displays an error. Initialization can also fail if existing history cannot be read or the initial high score cannot be saved. The full menu reappears after either an empty message or an error, keeping your selected difficulty.

Main menu and game difficulty

The main menu has six options. Enter a number or its name (names are case-insensitive):

1. Start
2. Records
3. Exit
4. Choose Difficulty
5. Continue
6. Leaderboard

Choose Difficulty opens a menu with `1. Easy`, `2. Normal`, `3. Hard` and `4. Coder`. Enter a number or difficulty name to select it and return to the main menu. Invalid input keeps you in the selection menu; `exit` quits the application.

Game difficulty	Player damage multiplier	Countdown multiplier	Starting HP
Easy	x4	x4	400
Normal	x2	x2	200
Hard (default)	x1	x1	100
Coder	x0.8	x0.8	80

The current difficulty appears in the main menu and at the start of a run. It applies throughout all five scenarios and both boss phases. Returning to the menu after winning or losing keeps the selection for your next run. Restarting the application defaults new runs to Hard; Continue uses the difficulty stored in the saved run, even if the menu currently shows another difficulty.

Starting health is multiplied once at the start of a run; health is not restored between fights. Enemy health and attacks are unchanged. Game difficulty does not change the passages selected by skills or your recorded WPM and accuracy.

Seeds and repeatable runs

A seed controls the run's random enemy and passage choices. After entering `start`, you can enter:

- A signed 64-bit whole number, such as `42` or `-19`. The game uses that exact number.
- Any nonblank text, such as `practice-run`. The same text is consistently converted to the same numeric seed.
- An empty line. The game generates and displays a random numeric seed.

Reusing a seed reproduces the same choices when you make the same decisions in the same order. Enemy selection and prompt selection share one random sequence. Choosing different skills, especially Encryption, consumes that sequence differently and can change later enemies or passages. A seed is intended for repeatable gameplay and testing, not for security.

The game also remembers where it is in that sequence whenever it autosaves. Continuing a current save preserves future enemy and passage choices as well as the rule that the same passage difficulty does not immediately repeat its previous passage. You do not need to enter the original seed again. Older saves created before random-state saving was added can still load, but their future random choices start from a new random sequence.

Saving and continuing a run

The game uses one save slot at `data/game-save.properties`, relative to the launch folder. It saves automatically at the start of every encounter and after the player attack and any enemy retaliation finish. The save contains your remaining HP, difficulty, scenario or boss phase, enemy identity and remaining HP, and the enemy's next attack in its rotation. It also preserves the random sequence position and previous passage selections.

To leave and return later:

1. Type `exit` to quit the application.
2. Launch the JAR again from the same folder, or use the same source launcher as before.
3. Enter `5` or `continue` at the main menu.

You return to skill selection without healing or an extra enemy attack. Progress saves automatically; no save command is needed.

Quitting or closing the terminal during typing retains the previous stable turn. The unfinished answer and timer are not saved, and you can select a skill again after continuing. Future random choices resume from that saved point, including the passage history used to avoid immediate repeats. With the same skill choices

and combat progression, future passages and enemies match an uninterrupted run. Older saves without this random-selection information still load their combat progress, but begin a fresh random sequence.

Start begins a new run and replaces the previous slot after encounter setup succeeds. Winning or losing deletes the active run save, while your typing history and high score remain available through Records and Leaderboard. Choosing a different difficulty in the menu does not alter a saved run; Continue restores its original mode.

All three data files are relative to the folder where you launch the application. Launching from a different folder can show different records, leaderboard and saved progress. Keep the `data` folder with your game folder when moving your saved data.

If no save exists, Continue displays a message and keeps you in the menu. Invalid, unsupported or unreadable saves display an error without starting a run. Saving failures are reported but combat continues; until a later save succeeds, the previous saved turn remains the latest resume point. A failure to remove a finished save is also reported.

Skills and countdown

Each turn begins with a skill choice. Names are case-insensitive.

Skill	Passage difficulty	Base minimum time limit (Hard)
`Ping`	Easy	15 seconds
`Brute Force`	Medium	30 seconds
`Buffer Overflow`	Hard	30 seconds
`SIGKILL`	Ultimate	45 seconds
`Encryption`	Ultimate (random special characters)	Exactly 60 seconds

`Encryption` generates a fresh random sequence using the symbols `!@#$%^&*()\_+/*{}|:"<>?`. Spaces separate groups of one to six characters. The passage is approximately the same length as a `SIGKILL` passage, with a base time limit of exactly 60 seconds. Its final countdown is 240 seconds on Easy, 120 on Normal, 60 on Hard or 48 on Coder.

For the other skills, the selected passage can increase that limit: the game allows enough time to type its normalized character count at 50 WPM, adds a 20% time buffer and rounds up to a whole second. It uses whichever is larger: that allowance or the skill's minimum. The game then multiplies that base time by the selected game difficulty's multiplier and rounds up to a whole second. Check the displayed time limit for each attempt.

In a supported terminal, the timer appears above "Type the following..." and updates while you type. Enter submits one attempt, including an imperfect or empty answer. If time expires first, the game captures your current input, clears the input line and scores the captured text. Corrections made before submission or expiry count; previously corrected mistakes do not affect the result.

Plain-text mode prints the time allowance and passage once, then waits for Enter. It does not redraw a countdown or submit an empty turn while the console is still holding your input. A late submission is evaluated using its actual elapsed time with the same accuracy and damage rules.

Every skill uses the same player damage formula. The skill changes the passage and its time allowance, so choose a difficulty suited to the typing practice you want. Enemy attacks use their own fixed skill damage.

Combat and progression

The game shows WPM, character accuracy, elapsed time, required time and attack damage. Individual wrong words are not printed. There is no separate damage deduction for exceeding the deadline; your captured text and actual elapsed time still determine speed and accuracy. Expiry is announced with "Time's up". If the enemy survives, it attacks before your next skill choice. A defeated enemy does not retaliate.

Encounter	Starting enemy HP	What happens after victory
Scenario 1	30	Start scenario 2
Scenario 2	35	Start scenario 3
Scenario 3	50	Start scenario 4
Scenario 4	60	Start scenario 5
Scenario 5	70	Start the final boss
Kernel Overlord, phase 1	50	Start phase 2; boss attacks deal 10 damage in phase 1
Kernel Overlord, phase 2	70	Win the run; boss attacks deal 20 damage in phase 2

Each regular scenario randomly selects an enemy name and attack rotation. Scenario number determines its starting HP. Passages are also random within the chosen difficulty, and the same difficulty does not repeat its previous passage immediately. Enemies and passages can reappear later.

Your remaining HP carries into each scenario and boss phase without healing. Damage beyond the current enemy's remaining HP is discarded: it does not carry into the next enemy or boss phase. Starting a new phase does not grant the boss an extra attack. Defeating phase 1 alone does not win.

On Hard, at 50 WPM and 95% character accuracy, each attack deals 40 damage. This is enough to defeat each regular enemy and each boss phase in at most two attacks. With that performance throughout a run, the strongest possible regular enemy choices leave you with at least 10 HP after victory. Slower or less accurate attempts can require extra turns and lead to defeat.

If your HP reaches zero, the run ends immediately. Victory or defeat returns to the main menu; `start` begins a new run at scenario 1 with the selected difficulty's starting HP.

Typing feedback

Words per minute (WPM)

WPM is gross typing speed, using five characters as one word:

```
WPM = (normalized submitted characters / 5) / (elapsed seconds / 60)
```

Leading and trailing whitespace is removed, and repeated internal whitespace becomes one space. The remaining letters, punctuation and spaces count toward WPM; Enter does not. Incorrect characters also count toward gross WPM, so read speed alongside accuracy. Empty or whitespace-only input gives 0 WPM.

Character accuracy

Accuracy uses the minimum number of single-character insertions, deletions and substitutions needed to change the normalized expected text into the normalized input:

```
Accuracy % = 100 * (1 - edit distance / longer normalized text length)
```

Capitalization and punctuation count. A single missing character reduces accuracy for that edit without shifting every later character into an incorrect position. Matching text gives 100%; empty input gives 0%. The game evaluates the final text rather than the sequence of keystrokes used to produce it.

Damage and the deadline

```
Damage = floor(WPM * (accuracy percentage / 100)^4 * game difficulty multiplier)
```

`floor` means round down to a whole number once, after applying the multiplier. On Hard, at 50 WPM and 100% accuracy, damage is 50. At 50 WPM and 95% accuracy, it is `floor(50 * 0.95^4) = 40`. Accuracy has a strong effect: at 50 WPM and 80% accuracy, damage is 20. This formula applies to every player skill.

At 50 WPM and 95% accuracy, damage is 162 on Easy, 81 on Normal, 40 on Hard or 32 on Coder.

Elapsed and required times must be positive, finite values. A result is overdue only when elapsed time is strictly greater than required time. The summary shows both times. Expiry is announced with "Time's up" rather than a separate status line. In a supported terminal, automatic expiry ends the attempt and can trigger the surviving enemy's turn. In plain-text mode, Enter ends the attempt even when it is late. Neither mode adds a separate damage penalty for exceeding the deadline. Displayed metrics use two decimal places; calculations use the full values.

Example

On Hard, for `Strike the goblin`, submitting `Strike teh goblin` in 6 seconds gives:

```
WPM: 34.00
Accuracy: 88.24%
Elapsed time: 6.00 seconds
Required time: 10.00 seconds
```

The swapped letters require two character edits. Damage is **20**, after applying the accuracy formula and rounding down. The correct text in the same six seconds gives **100.00% accuracy** and **34 damage**. Supplying a five-second deadline instead would mark the six-second result overdue; the same text and elapsed time would produce the same damage.